Keshav Ramphal Presents :

Cyber Security Task 1 (2026)

By Future Interns

Vulnerability Assessment Report for a Live Website (Read-Only Scope) on Altoro Mutual : http://altoro.testfire.net/login.jsp

Date : 24 February 2026

Task : 1

Table of Contents

Table of Contents (Updated) 📄

1. Introduction & Disclaimers ..... Page 3

- **Ownership and Intellectual Property Rights**
- **Legal Disclaimer & Ethical Hacking Standards**

2. Executive Summary ..... Page 5

- **Scope of Work: Altoro Mutual Analysis**
- **Project Timeline & Methodology**

3. Infrastructure & Reconnaissance ..... Page 6

- **Port Exposure Analysis (Nmap Scan Results)**
- **Security Header Checks**

4. Vulnerability Assessment ..... Page 8

- **High-Risk Alerts (SQL Injection, IDOR)**
- **Medium-Risk Alerts (XSS, Security Misconfigurations)**

5. Database Design & Security ..... Page 10

- **ERD Design Principles (Crows Foot Notation)**
- **Data Integrity & Leak Prevention**
- **Relationship Mapping (1:1, 1:M, M:M Bridge Tables)**

6. Corrective Actions & Recommendations ..... Page 12

- **Patching SQLi & XSS Vulnerabilities**
- **Implementing Primary & Foreign Key Constraints**

7. Conclusion & References ..... Page 14

Disclaimers and information related to the tasks conducted.

Ownership and Rights with this documents.

Any items and test conducted with in this document, legally belongs to Keshav Ramphal. In terms of all the research done in order to construct this document to any images and reports that where produced belong one aging to Keshav Ramphal. And if a third party with like to use the paper for research or personal use.

They should :

1. Not Plagiarism of this document.
2. If this document is in use : credit the writer of the paper being Keshav Ramphal.

Disclaimers and Information.

REMEBER : ANY form of accessing a system, whether it be hardware or software without legal and firm permission (Contacts or written agreements) from a person/ firm / website , is punishable by law and will land you in trouble legally and will result in possibly imprisonment or/and fines . And this vary in different country, where by some may be worse then others.

ALSO : The site that I've done testing on is a **open source website and provided for testing** out ethical testing skills by : Copyright © 2008, 2017, IBM Corporation, All rights reserved. Copyright © 2017, 2026, HCL Technologies, Ltd., All rights reserved.

And also provided on the website i quote : ‘The AltoroJ website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to https://www.hcl-software.com/appscan/.’

Timeframe

The timeframe of testing on the website was from : 20 Feb 2026 - 23 Feb 2026 , where i **DID NOT** conduct unethical hacking like DDoS, Hostile take over, scamming or the execution of scripts etc. I did however only test out log in, and if the firm has strong passwords and if they prepared any anti-SQL injection methods. And followed the scope that was provided by Future Internship which where :

- Public-facing pages only
- Passive scanning
- Header checks
- Configuration analysis

And also used the tools that Future Internships said to use, which where :

- **Nmap** – basic port & exposure analysis
- **OWASP ZAP (Passive Scan)** – identify vulnerabilities without attacking
- **Browser DevTools** – inspect headers, cookies, and client-side issues

The Report

Step 1 : Planning and Discovery's

1. Installing and setting up a virtual machine with Kali Linux on it in order to access the pre-installed tools that come with the set Linux distro. And plus i have some experience with the distro.

2. Getting what tools updated and running, which where ZAP (OWASP) and Nmap and browser tools came with the browser of course.

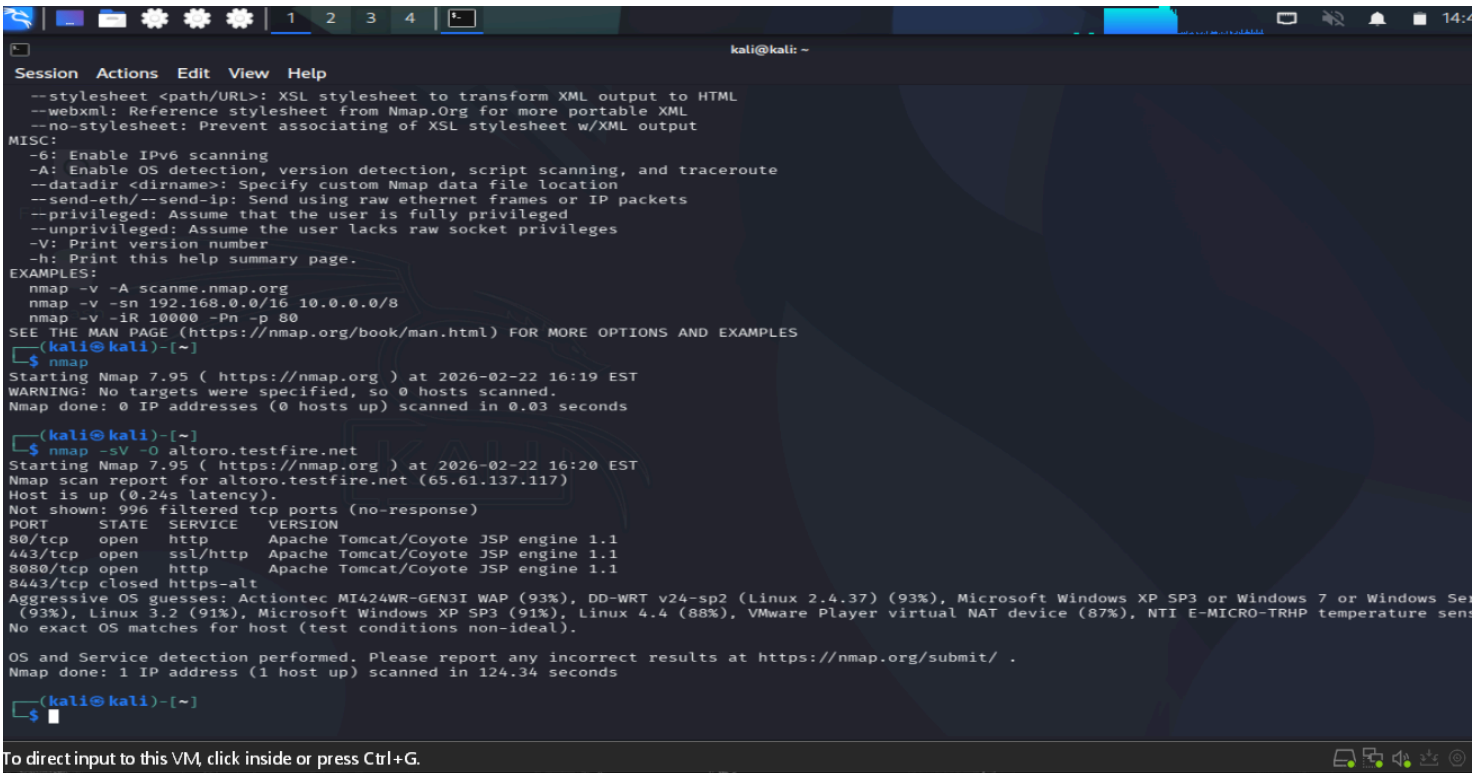
3. There after googling and researching for a website in order to do testing in along with analyzing the set guidelines to prevent crossing for example DDoS and Harmful Scripts.

4. There after i set my plan in terms of what tools i will use in a certain order to find any flaws in the website. And there after went onto the website in order to gain information like.

- Entry points : Like login box's and password box's.
- How old the Web page is
- If there is any suspicious items on the site page its self like there personal information and also anything about the set firm.

Step 2 : Attack and Reports found

1. First i used Nmap in order to find which ports where open along with what kind of hardware was used in order to host the website



as shown in the image is me in the Kali Linux V.M , I ran Nmap along with the command : nmap -Sv -O http://altoro.testfire.net .

What does the following command and also what was exposed and the possible damages ?

1.1 Nmap - is the program used in order to look for ports that are open with in a system or network and this could be both helpful or harmful depending on who is using the set tool and knows about the set information. When used in a command its like the root and activates and allows for other branch's to activate or combining it like in the above

could result in good or bad results.

1.2 -Sv - is a command used with Nmap in order to scan a set websites service version that is running on the port its self. In the example above it with be the Apache Tomcat

1.3 -O - is used in order to get any system information about the set equipment like the make and also what OS it could be possibly running. And all of it is conducted remotely.

What Vulnerability's where found ?

Port 443, 8080 and 80 are open, but exactly with be the problem.

For port 443 - this is one port which is ok to be open since it takes data in and it is secure but if the end if the pipeline is weak then threats can brake in via scripts and possible SQL injections.

For port 8080 - on the other hand is a developer port used when developing a web site and acts kind of like a back door for the devs and if hackers find port 80 hard to crack they offer resort to port 8080 to brake in due to less protection.

For port 80 - is bad when it comes to it being open because it means data is not secure and also hackers can take advantage of this via the three way hand shake in order to manipulate data and also steal it.

What are the possible solutions toward the open ports

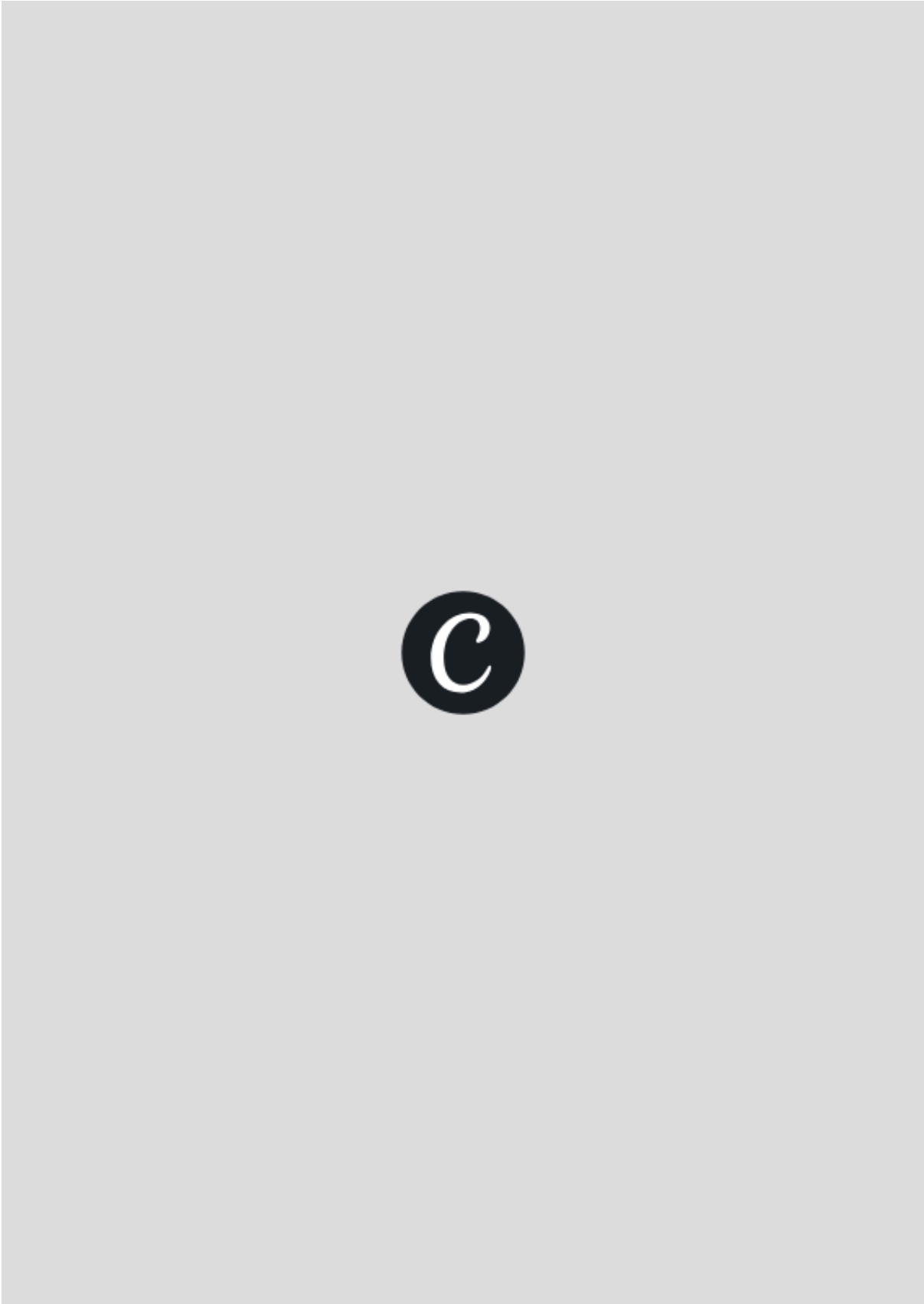
1.1.1 Is to close of Port 80 and also Port 8080 due to being direct attack vectors for hacks in terms that they can easily access data on the website or also break into sensitive

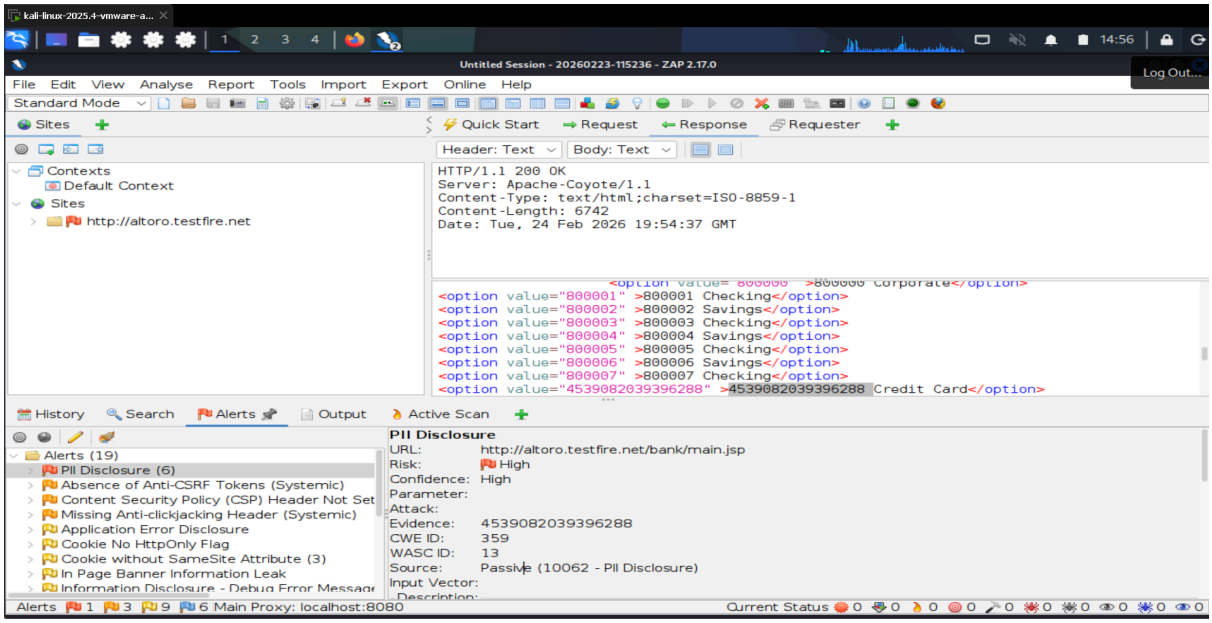
1.1.2 The implementation of HTTPS in order to encrypt data from outside threats and also prevents snooping on the three way hand shake.

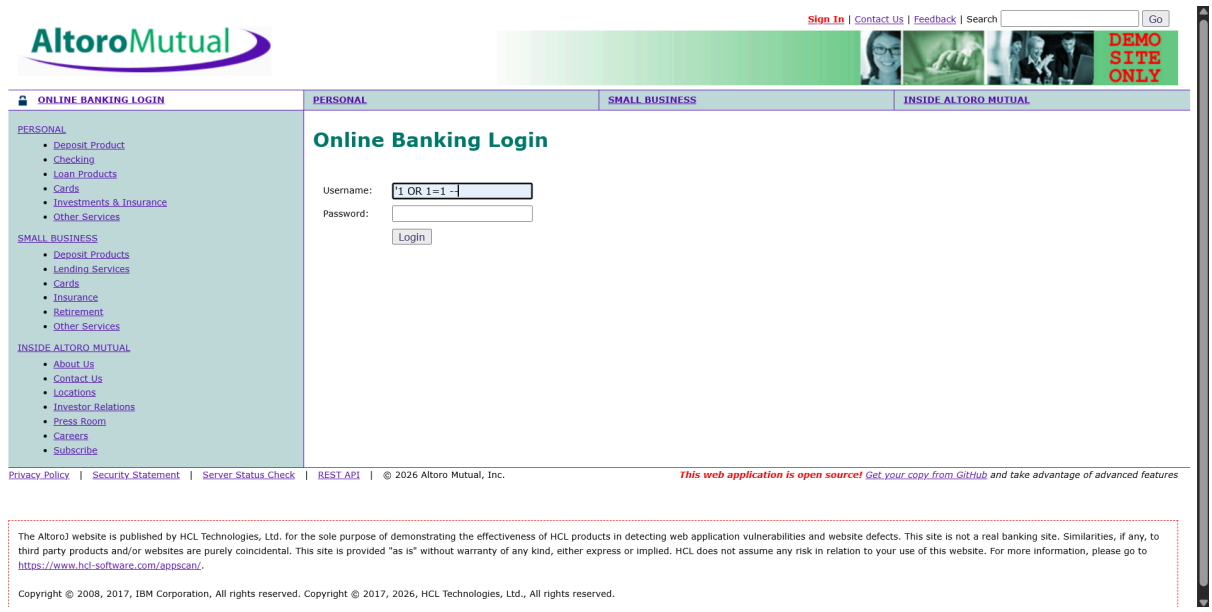
1.1. 3 Firewall the Dev Ports: If they *need* 8080 for testing, it should be behind a VPN, not open to the whole internet. and prevents from and cross platform cross manipulation.

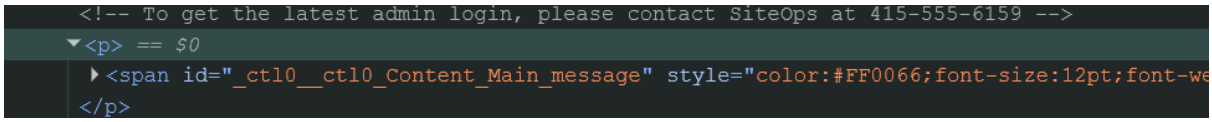
1.1.4 Updating of hardware the Coyote 1.1 is very outdated and has alot of security flaws like the : Ghostcat (CVE-2020-1938): Specifically for Apache Tomcat/Coyote (the hardware/software in your report), there is a famous exploit called "Ghostcat" that often lives on ports like 8080. It allows a hacker to read *any* file on the server, including WEB-INF/web.xml which contains database passwords.

2. I thereafter used Browser tools and looking at the code via inspect element and also Zap in order to get information









The scan flagged a mix of high and medium-risk alerts. Here's the split:

- **High Risk Alerts**: 4 total (1 High Confidence, 1 Medium, 2 Low).
- **Medium Risk Alerts**: 6 total (1 Medium Confidence, 5 Low).

Common Heavy Hitters for this site:

- **SQL Injection (SQLi)**: One of the biggest baldest bugs. Attackers can mess with your database, bypass logins, or steal sensitive info.
- **Cross-Site Scripting (XSS)**: Attackers can run malicious JavaScript on your users' browsers, often through the search box.
- **Insecure Direct Object Reference (IDOR)**: Users can basically peek at (or change) other people’s private data just by swapping out IDs in the URL/request.
- **Absence of Anti-CSRF Tokens**: Makes it possible for attackers to trick users into doing stuff they didn't mean to, like changing their password.

Full report on GitHub :
FUTURE_CS_01/This4-ZAP-Security-Report-Enhanced.html.pdf at …